Système et Réseaux : TP8
Couche transport

avril 2025 — Pierre Rousselin (conseils techniques : Thierry Monteil)

Exercice 1 : Couche liaison : interfaces liaison et adresses MAC

La commande `ip` (bien mal nommée car s'occupant à la fois de la couche réseau (IP) et de la couche liaison) permet en particulier de connaître les *interfaces* liaison du système.

1. Lister les interfaces à l'aide de la commande `ip link`[1].

2. Laquelle correspond à du `loopback` ? Quelle est son adresse MAC ?

3. Laquelle correspond au câble éthernet branché à la machine ? Si vous avez des doutes, utiliser la commande `ip -s link` pour lister aussi la quantité d'information envoyée et reçue.

4. Quelle est l'adresse MAC de *broadcast* sur cette interface ?

5. Sur Linux seulement. Lister les cartes PCI avec la commande `lspci`. Trouver la carte réseau, si celle-ci est bien branchée au bus PCI. Comparer les nombres affichés (en hexadécimal) de la forme `nn:mm.p` à ceux qui apparaissent (en décimal) dans le nom (ou l'*altname*) de l'interface affichée par `ip link`. Si besoin, `man 7 systemd.net-naming-scheme`.

Pour plus d'informations sur `ip link`, utiliser la commande `ip link help` ou bien `man ip link`.

--- * ---

Exercice 2 : Couche réseau : adresse IPv4

1. La commande `ip a`, ou `ip address`[2]donne les adresses IP de la machine sur les différents réseaux IP auxquels elle est connectée.

2. Quel est l'adresse IPv4 de loopback ?

3. Quel est l'adresse IPv4 sur le réseau correspondant au câble éthernet branché à la machine ? Si vous ne vous y retrouvez pas, vous pouvez utiliser la commande

```
$ ip address show dev <NOM_INTERFACE>
```

où `<NOM_INTERFACE>` est le nom de l'interface liaison trouvée dans l'exercice précédent.

4. Est-ce un réseau privé ou public ? Si besoin, aller voir `https://fr.wikipedia.org/wiki/Adresse_IP#Plages_d'adresses_IP_sp%C3%A9ciales`.

Pour plus d'informations sur `ip address` :

```
$ man ip address
```

--- * ---

Exercice 3 : Les ports

1. Afficher le contenu du fichier `/etc/services` pour répondre aux questions suivantes.

a) Quels sont les ports « bien connus » ?

b) Quels sont les ports privés, utilisables comme ports éphémères ?

c) Quels ports sont associés aux protocoles `ssh`, `http`, `dns` ? Pour sélectionner les lignes dont la première colonne est `blah`, on peut utiliser

```
$ awk '$1 == "blah"' /etc/services
```

2. a) Afficher la liste des `sockets` TCP, les ports associés et les processus avec la commande `ss -tp`. Voir les lignes associées à votre navigateur web (s'il n'est pas ouvert, ouvrez-le).

1. sur MacOS, on reste sur `ifconfig`, `if` comme *interface*.
2. Sur MacOS, utiliser encore `ifconfig`.

b) Certains des ports affichés sont des *well-known ports* et leur numéro est remplacé par un nom dans la sortie de la commande. Chercher rapidement dans le fichier `/etc/services` les numéros associés et les noms complets des protocoles. Se documenter brièvement sur le web sur ceux-ci pour savoir à quoi ils servent.

c) Les ports utilisés par votre navigateur sont-ils bien connus ?

d) Si vous ouvrez deux onglets différents vers le même serveur web, combien de sockets sont créées ?

3. À l'aide des commandes `dig` et `nmap` voir quels sont les sockets en écoute (et les ports associés) sur `ent.univ-paris13.fr`. À quels services correspondent-ils ?

--- * ---

Exercice 4 : `socat` sur une seule machine

1. Ouvrir deux terminaux. Dans le premier terminal, lancer la commande

```
$ socat -dd udp-recv:2222 -
```

Dans le second,

```
$ socat -dd - udp-send:127.0.0.1:2222
```

Puis entrer des messages. Voir sur le second terminal. Quel port éphémère est utilisé par le processus qui envoie des datagrammes ?

2. À quoi correspond l'adresse `127.0.0.1` ? Essayer d'envoyer un datagramme à `127.42.42.42` sur le port `2222`. Commenter.

3. En fait, toutes les adresses `127.0.0.1/8` sont des adresses de *loopback* et le processus `socat` qui reçoit est configuré pour recevoir les datagrammes UDP sur le port `2222`, quelle que soit l'adresse par laquelle il les reçoit, *ce qui est presque toujours ce que l'on veut et un bon comportement par défaut.* On dit qu'on a *bind* (attaché) l'entrée du processus au port 2222 et à l'adresse 0.0.0.0, c'est-à-dire tout ce qui arrive.

Ceci dit, on va s'amuser un peu avec les adresses de *loopback* : on peut *bind* plus précisément les entrées en restreignant les adresses de destinations.

Tuer tous les processus `socat`, puis en créer deux nouveaux qui reçoivent des datagrammes, le premier avec la commande

```
$ socat -dd udp-recv:2222,bind=127.42.42.42 -
```

Le second avec

```
$ socat -dd udp-recv:2222,bind=127.1.1.1 -
```

En utilisant des nouvelles commandes `socat`,

a) envoyer un datagramme au premier (vérifier que le second n'a rien reçu) ;

b) envoyer un datagramme au second (vérifier que le premier n'a rien reçu). [3]

4. Que donne la sortie de `socat udp-recv:25 -` ? Pourquoi ?

5. Que se passe-t-il si vous lancez deux fois la commande

```
$ socat -dd udp-recv:2222 -
```

--- * ---

Exercice 5 : `socat`, c'est mieux à plusieurs

1. (sur deux hôtes) Convenez d'un numéro de port pour le processus qui reçoit le premier datagramme. Puis l'un d'entre vous lance avec `socat` un processus qui reçoit des datagrammes sur ce port et l'autre lui en envoie.

Pour le moment vous ne pouvez pas vous répondre, c'est le moment idéal pour lui dire tout ce que vous avez sur le cœur sans qu'il/elle puisse vous interrompre !

Qu'est-ce qui doit être connu à l'avance pour que l'envoi du premier datagramme puisse avoir lieu ?

3. Je voulais aussi faire du *broadcast* mais visiblement ce n'est pas possible en *loopback*.

2. (sur deux hôtes) La commande `socat` permet en fait des échanges dans les deux directions.
Un premier processus attend (écoute) un premier message :

```
$ socat -dd udp-listen:port -
```

et le deuxième lui envoie un message et les deux processus seront *faussement connectés* :

```
$ socat -dd - udp:adresse:port
```

où `port` et `adresse` sont à remplacer convenablement.
Vous pouvez ensuite envoyer et recevoir des messages des deux côtés.

3. (sur au moins trois hôtes) Au moins deux processus sur au moins deux machines reçoivent des datagrammes sur le même port UDP.
D'autres processus envoient des datagrammes en *broadcast* sur ce réseau avec :

```
$ socat -dd - udp-send:adresse:port,broadcast
```

où `adresse` est à remplacer par l'adresse de *broadcast* du réseau et `port` par celui qui a été choisi.

--- * ---